

# Apêndice: o mesmo servidor, na nuvem

**Para quem:** você terminou a Aula 4, tem a API rodando na VM do seu notebook, e quer colocá-la num IP público, com domínio, certificado de CA pública e deploy automático a cada push.

**Quanto tempo:** ~2h na primeira vez, quase tudo clicando em portal.

**Quanto custa:** nada, com a [Azure for Students](#), US\$100 de crédito, sem cartão de crédito, mediante e-mail institucional. A verificação acadêmica **pode demorar dias**; comece por ela.

Nada aqui substitui a Aula 4. O Docker, o Kamal e o `deploy.yml` são **os mesmos**. O que este apêndice acrescenta é o que só existe quando a máquina não é a sua: provisionar um Ubuntu do zero, firewall de provedor, DNS, certificado de CA, e um jeito de o GitHub entrar na máquina sem guardar senha.

## O que muda

|                | Na Aula 4                                 | Aqui                                                                                      |
|----------------|-------------------------------------------|-------------------------------------------------------------------------------------------|
| A máquina      | a sua, já configurada                     | um Ubuntu cru, no portal da Azure                                                         |
| Provisionar    | nada: só instalar o <code>sshd</code>     | <code>apt upgrade</code> , <code>ufw</code> , Docker, swap, endurecer o <code>sshd</code> |
| IP             | <code>127.0.0.1</code>                    | público, e o mundo inteiro alcança                                                        |
| Firewall       | nenhum                                    | <code>ufw</code> <b>mais</b> o NSG da Azure                                               |
| Nome           | <code>/etc/hosts</code>                   | DNS de verdade, no Cloudflare                                                             |
| Certificado    | autoassinado                              | Cloudflare Origin CA, com o público do Cloudflare na frente                               |
| Deploy         | <code>kamal deploy</code> no seu terminal | GitHub Actions, por OIDC                                                                  |
| Usuário do SSH | o seu login                               | <code>azureuser</code>                                                                    |

Os dois últimos são duas linhas do `.env`:

```
export SERVER_IP=57.156.65.151      # em vez de 127.0.0.1
export KAMAL_SSH_USER=azureuser    # em vez do seu login
```

# 1. Criar a VM na Azure

## Cotas antes de tudo

A assinatura Azure for Students não oferece todos os tamanhos em todas as regiões. O portal responde:

NotAvailableForSubscription

O caminho certo:

1. **Assinaturas** → **Azure for Students** → **Uso + cotas**
2. filtre por `Compute`
3. veja as cotas regionais
4. escolha uma região onde o tamanho aparece

Não insista num tamanho marcado como indisponível. Pedir aumento de cota não resolve quando a oferta não está habilitada para a assinatura.

## O assistente

**Máquinas virtuais** → **Criar** → **Máquina virtual do Azure**

| Campo                  | Valor                                                                 |
|------------------------|-----------------------------------------------------------------------|
| Grupo de recursos      | Novo: <code>rg-capacita-&lt;seunome&gt;</code>                        |
| Nome                   | <code>vm-capacita-&lt;seunome&gt;</code>                              |
| Região                 | uma com cota disponível                                               |
| Imagem                 | Ubuntu Server 24.04 LTS: x64 Gen2                                     |
| Tamanho                | <code>Standard_B1s</code> (1 vCPU, 1 GiB) ou maior, se houver crédito |
| Autenticação           | Chave pública SSH                                                     |
| Usuário                | <code>azureuser</code>                                                |
| Tipo de chave          | Ed25519                                                               |
| <b>Portas públicas</b> | <b>Nenhuma</b>                                                        |

**"Nenhuma" é a escolha mais importante da tela.** O assistente, se deixado, cria uma regra liberando SSH para a internet inteira. Bots varrem a porta 22 do IPv4 todo, o dia todo. Vamos abrir a 22 só para o seu IP.

Rede:

- VNet e sub-rede: aceite os padrões

- IP público: Standard
- NSG: avançado (é o firewall. Vamos mexer nele)

Marque tudo com tags (`ambiente=capacitacao`, `dono=<seunome>`): é assim que você acha recurso órfão consumindo crédito depois.

## A chave privada

O portal oferece o download **uma vez**.

```
mkdir -p ~/.ssh
mv ~/Downloads/vm-capacita-seunome_key.pem ~/.ssh/azure-capacita
chmod 600 ~/.ssh/azure-capacita
```

`chmod 600` = só você lê. O SSH **recusa** usar uma chave com permissão frouxa.

## Abrir o SSH só para você

No NSG da VM, **Regras de segurança de entrada** → **Adicionar**:

| Campo             | Valor                         |
|-------------------|-------------------------------|
| Origem            | Endereços IP                  |
| IPs de origem     | <code>SEU_IP/32</code>        |
| Portas de destino | <code>22</code>               |
| Protocolo         | TCP                           |
| Prioridade        | <code>100</code>              |
| Nome              | <code>allow-ssh-meu-ip</code> |

Descubra o seu IP:

```
curl -4 ifconfig.me
```

O `/32` significa "exatamente este endereço". Precisa também de:

| Porta            | Para quê                             |
|------------------|--------------------------------------|
| <code>80</code>  | HTTP (o Cloudflare precisa alcançar) |
| <code>443</code> | HTTPS                                |

Internet de casa troca de IP. Quando o SSH parar de conectar do nada, é isso: atualize a regra.

**Este é o firewall que a sua VM local não tinha.** O `ufw` continua valendo dentro da máquina, e é uma segunda camada, mas o NSG é o que faz o pacote nem chegar.

## Primeiro acesso

```
ssh -i ~/.ssh/azure-capacita azureuser@SEU_IP_PUBLICO
```

O prompt vira `azureuser@vm-capacita`. **Você está dentro de outro computador**, e agora vem a parte que a Aula 4 não teve: deixar essa máquina pronta para servir.

## 2. Provisionar o Ubuntu

Na Aula 4 o servidor era a sua máquina, que já estava configurada. Aqui você recebe um Ubuntu cru, e tudo o que ele precisa você instala. **É este bloco que a nuvem acrescenta.**

### Atualizar

```
sudo apt update && sudo apt upgrade -y
test -f /var/run/reboot-required && sudo reboot
```

Numa máquina exposta isso não é higiene: é a correção das falhas que já foram descobertas desde a imagem ter sido publicada.

### Firewall

Uma máquina só deve aceitar conexão no que ela realmente serve. O Ubuntu traz o **ufw** (*Uncomplicated Firewall*), uma casca amigável sobre as regras do kernel.

```
sudo ufw default deny incoming      # nada entra...
sudo ufw default allow outgoing     # ...mas a máquina pode sair
sudo ufw allow 22/tcp                # SSH
sudo ufw allow 80/tcp                # HTTP
sudo ufw allow 443/tcp              # HTTPS
sudo ufw enable
sudo ufw status verbose
```

**Libere a 22 antes do `enable`**. Habilitar o firewall com a 22 fechada, numa máquina remota, é o jeito mais rápido de perder o acesso a ela.

Aqui há **dois** firewalls: o `ufw` dentro da máquina e o NSG do provedor, fora dela. O de fora é o que importa mais, porque com ele o pacote nem chega. O `ufw` é a segunda camada, para o caso de a primeira estar mal configurada.

### Docker, do repositório oficial

O `apt install docker.io` do Ubuntu instala uma versão velha. Use o repositório da Docker:

```
sudo apt install -y ca-certificates curl
sudo install -m 0755 -d /etc/apt/keyrings
sudo curl -fsSL https://download.docker.com/linux/ubuntu/gpg -o /etc/apt/keyrings/docker.asc
sudo chmod a+r /etc/apt/keyrings/docker.asc

sudo tee /etc/apt/sources.list.d/docker.sources >/dev/null <<EOF
Types: deb
URIs: https://download.docker.com/linux/ubuntu
Suites: $(. /etc/os-release && echo "${UBUNTU_CODENAME:-$VERSION_CODENAME}")
Components: stable
Architectures: $(dpkg --print-architecture)
Signed-By: /etc/apt/keyrings/docker.asc
EOF

sudo apt update
sudo apt install -y docker-ce docker-ce-cli containerd.io docker-buildx-plugin docker-compose-plugin

sudo usermod -aG docker azureuser
exit
```

**Saia e entre de novo:** grupo só vale em sessão nova.

```
ssh -i ~/.ssh/azure-capacita azureuser@SEU_IP 'docker run --rm hello-world'
```

## Root e permissões

`root` é o usuário que pode tudo, sem "tem certeza?". Você trabalha como `azureuser` e chama `sudo` quando precisa. Numa máquina exposta essa separação deixa de ser estilo: se alguém escapar da aplicação, cai num usuário sem privilégio.

É a mesma razão de o Dockerfile criar um usuário `rails` e não rodar como root.

## Swap

```
ssh -i ~/.ssh/azure-capacita azureuser@SEU_IP 'sudo bash -s' < scripts/server-swap.sh
```

Uma `B1s` tem 1 GiB de RAM. Rails e Postgres juntos estouram isso, e o kernel mata o processo que estiver na frente (o *OOM killer*), com uma mensagem que não explica nada. 2 GiB de swap resolvem.

Na Aula 4 isso não fazia falta, porque o seu notebook tem RAM sobrando. Aqui faz.

## Endurecer o SSH

**Mantenha a sessão atual aberta** enquanto testa em outro terminal.

```
sudo tee /etc/ssh/sshd_config.d/99-hardening.conf >/dev/null <<'EOF'
PermitRootLogin no
PasswordAuthentication no
KbdInteractiveAuthentication no
PubkeyAuthentication yes
EOF

sudo sshd -t          # valida a sintaxe ANTES de recarregar
sudo systemctl reload ssh
```

Em outro terminal:

```
ssh -i ~/.ssh/azure-capacita azureuser@SEU_IP 'echo OK'
```

Só feche a primeira sessão depois que isso responder. Errar a config do SSH com uma sessão só aberta é o jeito clássico de perder acesso à máquina. E aqui, diferente da Aula 4, **você não está sentado na frente dela**: só sobra o Serial Console do portal da Azure, e é bom não depender dele.

## E então, o deploy

Daqui em diante é a Aula 4 sem mudar nada. No seu `.env`, duas linhas:

```
export SERVER_IP=57.156.65.151    # o IP público da VM
export KAMAL_SSH_USER=azureuser
```

E `kamal setup`.

---

## 3. Cloudflare e TLS

### Por que uma CA pública

Na Aula 4 o certificado era autoassinado, e o `curl` reclamava até você passar `--cacert`. Aqui você tem um domínio de verdade: dá para **provar** que ele é seu, e é isso que uma CA pública exige em troca de um certificado em que o mundo já confia.

### DNS

No Cloudflare, no seu domínio:

| Campo    | Valor                          |
|----------|--------------------------------|
| Tipo     | A                              |
| Nome     | seunome.capacita               |
| Conteúdo | o IP público da sua VM         |
| Proxy    | <b>Proxied</b> (nuvem laranja) |

**Proxied** significa que o tráfego passa pelo Cloudflare antes de chegar na sua VM. Você ganha cache, proteção contra DDoS e, o principal aqui, o IP real da sua máquina não aparece no DNS.

Agora há **dois** proxies reversos no caminho:

```
navegador → Cloudflare → kamal-proxy (na sua VM) → Rails
                (proxy 1)         (proxy 2)
```

## Certificado Origin CA

SSL/TLS → **Origin Server** → Create Certificate. O Cloudflare gera o par e mostra **uma vez**.

Ele protege o trecho **Cloudflare** → **sua VM**. O certificado que o *usuário* vê é o público do Cloudflare: o Origin CA nunca aparece para o navegador, e por isso não precisa ser confiável por ele.

É o mesmo lugar do `.env` que o autoassinado ocupava:

```
KAMAL_PROXY_SSL_CERTIFICATE="$(cat origin-ca.pem)"
KAMAL_PROXY_SSL_PRIVATE_KEY="$(cat origin-ca-key.pem)"
```

## Full (strict)

SSL/TLS → Overview → **Full (strict)**.

| Modo                 | Cloudflare → sua VM                            |
|----------------------|------------------------------------------------|
| Flexible             | <b>em texto puro:</b> não use                  |
| Full                 | criptografado, mas aceita certificado inválido |
| <b>Full (strict)</b> | criptografado e o certificado é validado       |

Com `Flexible`, o cadeado aparece para o usuário e a senha dele trafega em claro no último trecho. É pior que não ter HTTPS, porque mente.

## Por que não Let's Encrypt

O Kamal sabe emitir Let's Encrypt sozinho. Atrás do Cloudflare proxied não dá:

- o desafio HTTP-01 não chega ao seu servidor como o Let's Encrypt espera;

- daria para tirar o proxy durante a emissão, mas isso expõe o IP e vira ritual manual a cada renovação.

**Sem Cloudflare na frente**, com o DNS apontando direto para a VM, o Let's Encrypt é o caminho mais simples e o Kamal cuida sozinho:

```
proxy:
  ssl: true
  host: <%= ENV.fetch("APP_HOST") %>
```

## 4. OIDC: deploy sem senha

O jeito comum seria criar um *client secret* na Azure e guardar como secret do GitHub. Problemas: ele é longo, vale por meses, e quem tiver acesso ao repositório tem acesso à sua Azure.

**OIDC** (OpenID Connect) inverte: o GitHub emite, a cada execução, um token de curta duração que diz "sou o workflow do repositório X, na branch Y". A Azure verifica e devolve um acesso temporário.

**Não existe segredo de longa duração em lugar nenhum.**

### Criando

**1. Registro de aplicativo.** Em Microsoft Entra ID → Registros de aplicativo → Novo registro:

- nome: `github-capacita-<seunome>`
- contas: somente este diretório
- URI de redirecionamento: vazia
- **não crie segredo do cliente**

Anote: Application (client) ID, Directory (tenant) ID, Subscription ID.

**2. Credencial federada:** no app criado, Certificados e segredos → Credenciais federadas → Adicionar → cenário **GitHub Actions**:

| Campo            | Valor                 |
|------------------|-----------------------|
| Organização      | seu usuário do GitHub |
| Repositório      | nome do repositório   |
| Tipo de entidade | Branch                |
| Branch           | <code>main</code>     |

O subject resultante:

```
repo:SEU-USUARIO/SEU-REPO:ref:refs/heads/main
```



Precisa bater **caractere por caractere** com o que o GitHub emite. Erro de maiúscula, de nome ou de branch dá `AADSTS700213`, e a mensagem não ajuda.

**3. Permissão mínima:** no **NSG** (não na assinatura), IAM → Adicionar atribuição de função → **Colaborador de Rede** → o app criado.

Escopo no NSG e não na assinatura: se a credencial vazar, o estrago se limita a regras de firewall daquela máquina.

**4. Chave SSH de deploy.** Separada da sua chave pessoal:

```
ssh-keygen -t ed25519 -f ~/.ssh/capacita-github -C "github-actions" -N ""  
  
ssh -i ~/.ssh/azure-capacita azureuser@SEU_IP \  
'umask 077; mkdir -p ~/.ssh; cat >> ~/.ssh/authorized_keys' < ~/.ssh/capacita-github.pub
```

Chave separada dá para revogar o acesso do CI sem trocar o seu.

## 5. O workflow de deploy

`.github/workflows/deploy.yml` roda a cada push na `main`. O ponto delicado é o SSH: a porta 22 está fechada para a internet, e o runner do GitHub tem IP diferente a cada execução: não dá para liberar antes.

1. autentica na Azure (OIDC)
2. descobre o próprio IP público
3. cria uma regra no NSG liberando a 22 só para esse IP/32
4. faz o deploy
5. APAGA a regra – mesmo se o deploy falhar

O passo 5 tem `if: always()`. **Deixar a 22 aberta é o erro que transforma um deploy ruim num incidente de segurança.**

E aqui o build volta a acontecer no runner, e não no seu notebook. É o que você quer: o deploy deixa de depender da sua máquina estar ligada.

### Cadastrando no GitHub

**Settings** → **Secrets and variables** → **Actions**

**Secrets:**

| Nome                                                                                | Conteúdo                                                    |
|-------------------------------------------------------------------------------------|-------------------------------------------------------------|
| <code>AZURE_CLIENT_ID</code>                                                        | Application (client) ID                                     |
| <code>AZURE_TENANT_ID</code>                                                        | Directory (tenant) ID                                       |
| <code>AZURE_SUBSCRIPTION_ID</code>                                                  | Subscription ID                                             |
| <code>AZURE_SSH_PRIVATE_KEY</code>                                                  | conteúdo de <code>~/.ssh/capacita-github</code> (a privada) |
| <code>RAILS_MASTER_KEY</code>                                                       | conteúdo de <code>config/master.key</code>                  |
| <code>AUTOMIC_AUTH_API_DATABASE_PASSWORD</code>                                     | invente uma senha longa                                     |
| <code>JWT_SECRET</code>                                                             | saída de <code>bin/rails secret</code>                      |
| <code>SMTP_ADDRESS</code> / <code>SMTP_USERNAME</code> / <code>SMTP_PASSWORD</code> | do provedor de e-mail                                       |
| <code>KAMAL_PROXY_SSL_CERTIFICATE</code>                                            | certificado Origin CA                                       |
| <code>KAMAL_PROXY_SSL_PRIVATE_KEY</code>                                            | chave do Origin CA                                          |

Repare que **não existe** `KAMAL_REGISTRY_PASSWORD`: dentro do Actions, o `GITHUB_TOKEN` que o próprio GitHub gera já autentica no ghcr.io. O PAT que você criou na Aula 4 só era necessário porque você estava fora dele.

**Variables** (não são segredo):

| Nome                              | Exemplo                                    |
|-----------------------------------|--------------------------------------------|
| <code>GHCR_USER</code>            | seu usuário do GitHub                      |
| <code>SERVER_IP</code>            | <code>57.156.65.151</code>                 |
| <code>KAMAL_SSH_USER</code>       | <code>azureuser</code>                     |
| <code>SERVER_ARCH</code>          | <code>amd64</code>                         |
| <code>APP_HOST</code>             | <code>seunome.capacita.exemplo.tech</code> |
| <code>AZURE_RESOURCE_GROUP</code> | <code>rg-capacita-seunome</code>           |
| <code>AZURE_NSG_NAME</code>       | nome do NSG                                |
| <code>CORS_ORIGINS</code>         | <code>http://localhost:5173</code>         |
| <code>MAILER_FROM</code>          | Capacitação <noreply@exemplo.tech>         |

Pelo CLI (não deixa o valor no histórico do shell):

```
gh secret set JWT_SECRET
gh secret set KAMAL_PROXY_SSL_CERTIFICATE < origin-ca.pem
gh variable set APP_HOST
```

## O primeiro deploy

Actions → Deploy (Kamal) → Run workflow → command: `setup`

`setup` instala o Docker se faltar, sobe os accessories e faz o primeiro deploy. Só na primeira vez, depois é `deploy`, e ele acontece sozinho a cada push.

```
curl https://seunome.capacita.exemplo.tech/api/v1/status
```

```
{"status":"ok","service":"atomic-auth-api","environment":"production"}
```

E desta vez sem `--cacert`: o certificado que o seu `curl` recebeu foi assinado por uma CA em que ele já confiava.

## 6. Antes de ir embora

- [ ] **Desligar a VM** no portal (`Parar`). Parada, ela não consome crédito de computação. Uma `B1s` ligada 24h custa ~US\$8/mês: os US\$100 dão quase um ano, se você desligar.
- [ ] Conferir que a regra temporária de SSH sumiu do NSG:

```
az network nsg rule list --resource-group SEU_RG --nsg-name SEU_NSG --query "[].name" -o tsv
```

Não pode ter `allow-github-actions-deploy-ssh` na lista.

## Recapitulando

- O trabalho é o mesmo. O que a nuvem acrescenta é **exposição**, e cada peça nova aqui existe por causa dela.
- O firewall do provedor vem antes do `ufw`: o pacote nem chega.
- CA pública exige prova de domínio. É a única diferença real entre ela e o seu `openssl`.
- Full (strict), sempre. `Flexible` mente para o usuário.
- OIDC troca segredo de longa duração por token de curta. Prefira sempre.
- A porta 22 abre por um minuto e fecha: inclusive quando dá errado.

Os erros que realmente acontecem neste percurso estão em `troubleshooting.md`.